EXECUTIVE TECHNICAL BLUEPRINT

B2B Client Acquisition Automation Pipeline

A Principal Architect's Critical Analysis, Risk Teardown and Production Implementation Specification

PREPARED FOR

Granjur Technologies

Offshore software engineering · Custom development · MVPs · Staff augmentation

ENGAGEMENT	Self-hosted n8n orchestration architecture
SCOPE	Discovery · Enrichment · AI qualification · Compliant outreach
DOCUMENT DATE	24 June 2026
CLASSIFICATION	Confidential - Internal strategy & engineering
ENGINE	Self-hosted n8n + PostgreSQL + custom headless scrapers

Advisory note: This blueprint contains direct, opinionated engineering and growth recommendations. Several findings revise or override

the source plan where it carries material legal, deliverability, or platform-ban risk.

Contents & Executive Summary

EXECUTIVE SUMMARY

The plan is directionally strong: targeting intent-rich companies, decoupling the monolith, leaning on n8n for orchestration, and outsourcing deliverability to Instantly/Smartlead are all correct instincts. This blueprint does not restate those wins - it pressure-tests the plan and supplies the missing engineering specification. Four conclusions drive everything that follows.

- **Compliance is the existential risk, not bans.** A single GDPR complaint or a spam-trap (honeypot) hit can blocklist every Granjur outreach domain globally. Treat lawful basis, suppression, and list hygiene as P0 infrastructure, not a "later work" bullet.
- **Automating Upwork/Fiverr/LinkedIn messaging must be deleted from scope.** It is an account-death-sentence with near-zero upside. Replace it with a Human-in-the-Loop Slack/Discord queue - same intent signal, zero platform risk.
- **Data decays fast and silently.** B2B contact data rots at roughly 22-30% per year; scraped role data faster. Without confirmation/validation windows and re-verification, you will be emailing dead mailboxes and spam traps within months.
- **Keep the custom surface tiny.** Only Phase-1 discovery belongs behind rotating residential proxies. Everything downstream (enrichment, scoring, translation, sending) should be outbound API calls orchestrated by n8n - safe, observable, and restartable.

SECTION 01

Strategic Advisory & Hidden Blind Spots

Before architecture, growth strategy. These are the high-leverage layers absent from the current layout - the things that decide whether this pipeline compounds or quietly burns your domains and your TAM.

1.1 High-Leverage Strategic Tips (What You Missed)

Tip 1 - Instrument the funnel before you scale it: define your reverse funnel math

The plan optimizes volume ("maximum reach on all platforms, 24/7") before it defines a single conversion ratio. That is backwards. You cannot tune a machine you cannot measure. Lock a reverse-funnel model first and store every transition in the database so the numbers are real, not guessed.

Work backwards from revenue. If Granjur needs 4 new contracts/month at a 25% close rate on booked calls, you need 16 calls; at a 3% positive-reply-to-call rate you need ~530 positive replies; at a 5% reply rate that is ~10,600 verified, qualified contacts sent per month. Every stage below becomes a measurable target with an owner and an alarm threshold:

Funnel stage	Working assumption	Monthly target	Primary lever
Verified, qualified sends	top of funnel	~10,600	discovery + enrichment yield
Delivered (inbox, not spam)	>=95% delivered	~10,070	domain/list hygiene
Positive replies	~5% reply, 60% positive	~300	ICP fit + copy
Booked calls	~3% of positives	~16	offer + calendar friction
Closed contracts	~25% close	4	sales motion, not the bot

HIGH-LEVERAGE TIP

Make the database the source of truth for funnel math.

Every status transition in Section 2's schema is timestamped. A nightly n8n job rolls those into a cohort table so you see decay and conversion per source, per segment, per region - and can kill a channel that does not pay for its proxy cost.

Tip 2 - Lead with a 'reason to reach out', not a service menu

Granjur's offer is a commodity from the prospect's chair: every offshore agency claims React, mobile, and staff-aug. The plan's intent signals (hiring, funding, broken app, low Lighthouse) are gold - but only if the opening line references the specific observed trigger. The pitch is not "we do custom software"; it is "your checkout scores 31 on mobile Lighthouse and your LCP is 6.2s - here is the revenue math on that". Personalization keyed to a real, verifiable observation is the single biggest reply-rate multiplier and the strongest spam-filter defense you have.

Tip 3 - Separate the sending identity from the brand identity (already half-done)

The plan's lookalike domains (granjurtech.com, getgranjur.com) are correct, but the reasoning is incomplete. The point is not just "spares" - it is blast-radius isolation. Your primary granjur.com domain must NEVER send cold volume; it is your reputation anchor for warm replies and your website. Cold sending happens only on burner domains with their own mailboxes, their own warmup, and capped volume (~20-40 sends/mailbox/day). If a burner domain gets blocklisted, you rotate it out and your real brand is untouched.

This is covered operationally in Section 3.

Tip 4 - Add a feedback loop from won/lost deals back into the qualifier

The AI qualifier in Phase 3 will start with your hand-written heuristics and will be wrong a lot. Close the loop: when a deal is won, lost, or a reply says "not a fit", write that outcome back onto the lead row. Monthly, feed the won vs. ghosted cohorts back into the qualifier's prompt as few-shot examples. Within two or three cycles the model learns Granjur's *actual* ICP instead of your assumed one - the difference between a list that looks qualified and one that buys.

ARCHITECT'S NOTE

Channel that does not exist yet but should: warm referral & partner loops.
Agency directories and "partnership opportunities" are in the source plan but unspecified. Non-competing agencies (design studios, marketing shops) that get dev requests they cannot service are a higher-trust, lower-volume channel than cold email. Model them as a distinct source with their own (lighter-touch, never-automated) sequence - they convert at multiples of cold.

1.2 Data Degradation: Leads Go Stale, Quietly and Expensively

Scraped B2B data is a depreciating asset. The plan treats discovery as a one-time fill; in reality your database is leaking accuracy every single day.

Industry-observed decay for B2B contact data runs ~22-30% per year on email validity and faster on job titles (people change roles constantly). A founder email scraped in January may be a dead mailbox - or worse, a recycled address that has become a spam trap - by the time Phase 4 sends in April. Emailing decayed data is how good domains die: high bounce rates and trap hits are exactly the signals mailbox providers use to blocklist a sender.

Confirmation & validation windows (concrete policy)

Encode freshness as data, not as hope. Every lead row carries verification timestamps and a computed staleness state; n8n enforces the windows below.

Data element	Re-verify within	Action when stale	Hard stop
Email syntax + MX	30 days pre-send	Re-run verifier (ZeroBounce/NeverBounce)	Suppress on 'invalid'
Email deliverability	7 days pre-send	Re-validate; risky->skip	Never send 'risky/catch-all' raw
Decision-maker / title	90 days	Re-enrich via Apollo/Clay	Disqualify if role gone
Intent signal (job post)	14 days	Re-check; expired->cool down	Drop from hot queue
Company exists / domain	180 days	Re-resolve domain + HTTP 200	Archive if dead

FIX

Two non-negotiable gates before any address is queued for outreach.
Gate 1 - Syntax/MX/SMTP validation at enrichment time (catch-all and role addresses flagged, never sent raw). Gate 2 - A real-time re-validation in the 7-day window immediately before Phase 4 hands the lead to Instantly/Smartlead. A lead that fails either gate moves to SUPPRESSED, not back into the queue. Target a verified-list bounce rate under 2%; above 3% pause the sending domain automatically.

1.3 International Compliance: Keep Your Domains Off the Global Blocklist

This is the section the plan most dangerously under-weights. Compliance is not legal boilerplate - it is the deliverability and survival layer. Get it wrong once and every Granjur domain can be globally blocklisted, with no appeal.

GDPR - EU / UK / EEA corporate outreach

- **Lawful basis = legitimate interest (Art. 6(1)(f)), and you must document it.** B2B cold email to a relevant business role can qualify, but only with a recorded Legitimate Interest Assessment (LIA): why the contact is relevant, why the impact on them is minimal, and how they can object.
- **Personal data includes name@company.com and a person's title.** Scraping and storing it triggers GDPR. You owe transparency (who you are, why you have their data, a privacy link), and you must honor objection/erasure within 30 days - automatically.
- **Right to object must be one click and must actually delete.** Every email needs a working unsubscribe that writes the contact into a permanent suppression table keyed by hashed email AND domain. Re-importing a suppressed contact later must be blocked at the database layer.
- **National overlays are stricter than GDPR alone.** Germany (UWG) effectively demands prior consent for commercial email even B2B; France/CNIL is aggressive. Treat DE/AT as opt-in-only regions and route them to LinkedIn/manual channels, not cold mail.

> WATCH-OUT
>
> **Practical GDPR posture for Granjur: minimize, document, honor.**
> Store only what you use; attach a source + scrape-date + LIA reference to every EU lead; expose a privacy page; and make suppression irreversible. The DPA-meaningful fields (consent_basis, source, suppressed_at, erasure_requested_at) are in the Section 2 schema for exactly this reason.

CAN-SPAM - United States outreach

The US is permissive (no prior consent required for B2B cold email) but the rules are bright-line and the fines are per-email (up to ~$53,000 each). All of these are cheap to automate and expensive to skip:

- **No false or misleading headers.** From/Reply-To/routing must accurately identify Granjur (or the clearly-attributable sending brand). No spoofing the domain.
- **No deceptive subject lines.** The subject must reflect the message - no "Re:" on a first-touch cold email, a classic trick that filters now penalize hard.
- **Clear identification + a valid physical postal address.** A real mailing address in the footer is mandatory, including for the offshore entity.
- **Honor opt-outs within 10 business days; no fee, no hoops.** And never sell or transfer an address that opted out. Same suppression table as GDPR satisfies both.

Honeypots, spam traps & the blocklist kill-chain

Spam traps are addresses that exist only to catch senders who do not practice list hygiene. Pristine traps are seeded addresses that were never opt-in; recycled traps are abandoned real mailboxes a provider has reactivated as traps. Hit a few and Spamhaus / SpamCop / UCEProtect can list your sending IP or domain - which can cascade to a shared-IP blocklist that poisons unrelated senders too.

FIX

Anti-trap defense is procedural, and every step maps to this pipeline.

1) Never email scraped data without real-time SMTP verification (traps don't pass clean validation patterns and verifiers flag many). 2) Suppress role accounts (info@, sales@, admin@, postmaster@) - prime trap territory. 3) Never email an address that has bounced once. 4) Warm up every mailbox and cap daily volume so behavior looks human. 5) Monitor blocklists (e.g. via a monitoring API) and auto-pause a domain on the first listing. 6) Keep cold volume off granjur.com entirely.

RISK

The plan's instinct to 'maximize reach 24/7 on all channels' is the fast path to a global blocklist.

Volume without verification, throttling, and suppression is precisely the footprint blocklist operators hunt. Reach is a function of *deliverability x relevance*, not raw send count. Section 3 converts this principle into throttles and the channel-waterfall.

1.4 Localized Market Fatigue & Dynamic Intent Signals

Google Maps is a finite well. For a niche like "logistics companies in Riyadh" you will scrape the entire addressable set in a few passes - and then your 24/7 bot is re-discovering the same depleted list while annoying the same owners.

Two failures hide here. First, supply exhaustion: a city x niche cell has a hard ceiling of businesses, and once contacted they enter a cooldown - re-hitting them is fatigue, not reach. Second, staleness masquerading as discovery: the bot "finds" results but they are all already in the database. You need the pipeline to know when a cell is depleted and to pivot automatically.

Depletion accounting (treat each city x niche as an inventory cell)

- **Track saturation per cell.** Store a discovery_cells table: (region, niche, total_seen, new_last_run, contacted, cooldown_until). When new_last_run / total_seen drops below ~5% across two consecutive runs, mark the cell DEPLETED and stop scheduling it.
- **Rotate the search space, don't re-scrape it.** Expand by adjacent niche (logistics -> freight forwarding -> customs brokers), by geography (tier-1 -> tier-2 cities), and by language query (run the niche term in Arabic/Chinese to surface businesses English queries never return).
- **Enforce a per-company cooldown.** Once contacted, a company is locked for N days (e.g. 90) regardless of which cell re-surfaces it - dedup on company identity, not on the row.

Replace static scraping with dynamic, event-based intent

The durable fix for fatigue is to stop relying on a static directory and start reacting to events. Intent signals refresh themselves daily and self-target the highest-propensity buyers - the companies the source plan already named as "10x more likely to buy." Build these as recurring n8n-triggered sources:

Intent signal	Source / how to capture	Why it beats static Maps
Active dev job posts	Job boards / LinkedIn jobs / Indeed RSS by keyword	Proven hiring intent, dated, self-refreshing
Fresh funding rounds	Crunchbase-style feeds, press RSS	New budget + urgency to scale

Intent signal	Source / how to capture	Why it beats static Maps
Tech-stack change	BuiltWith / Wappalyzer change alerts	Migration = active dev project
Poor mobile perf	Lighthouse score < 50 on target sites	Concrete, quantified pain to open with
New product launch	Product Hunt / news mentions	MVP + scale need window
Job post expired+rehiring	Re-posts of same role	Failed to hire locally -> offshore fit

ARCHITECT'S NOTE

Reframe the whole top of funnel: Maps is a baseline source, intent feeds are the growth engine.

A depleted city x niche cell is not a dead end - it is a signal to shift budget from discovery scraping toward intent feeds, which never deplete because new jobs are posted, new rounds close, and new sites regress every day.

SECTION 02

Architectural Blueprint & Production DB Schema

The source plan's best architectural instinct - kill the monolith, decouple via a database and event triggers - is exactly right. Here is the production-grade version: the stack, the lead state machine, and executable PostgreSQL DDL.

2.1 Infrastructure Stack

Design principle: a tiny, hardened custom surface (discovery only) feeding a large, observable, restartable orchestration layer (everything else). n8n is the spine; PostgreSQL is the single source of truth; specialist SaaS handles the parts that are regulated, ban-prone, or simply not worth building.

Layer	Choice	Role & rationale
Compute host	1x AWS EC2 / DO droplet (2-4 vCPU)	Self-hosted n8n + Postgres via Docker Compose; ~$20-40/mo
Orchestration	n8n (self-hosted, queue mode)	Decoupled workflows; BullMQ/Redis queue mode for concurrency
Primary datastore	PostgreSQL 16 (or Supabase)	Single source of truth + JSONB for flexible enrichment payloads
Queue / async	Redis (n8n queue mode)	Worker concurrency, retries, rate-limited execution
Discovery scrapers	Custom Node/Python, headless	Behind rotating residential proxies; the ONLY custom surface
Email verification	ZeroBounce / NeverBounce API	Syntax+MX+SMTP; trap & catch-all detection
Enrichment APIs	Apollo / Clay / Hunter / BuiltWith	Decision-makers, firmographics, tech stack - outbound only
Performance API	Google PageSpeed (Lighthouse)	Mobile score as Segment-C intent signal
AI qualification	Claude / GPT via n8n AI nodes	Qualifier + localized translation chains
Sending / cadence	Instantly.ai or Smartlead.io	Warmup, timezone, follow-ups, A/B - never hand-rolled
Scheduling	Cal.com or Calendly (API)	Timezone overlap to PKT working hours
HITL + alerts	Slack / Discord (webhook)	Human-in-the-loop platform actions + node-failure alerts
Monitoring	Blocklist + uptime monitor	Auto-pause a domain on first listing

Topology (decoupled, event-driven)

TEXT · High-level event-driven topology

```
[ Custom Discovery Scrapers ]        (headless Node/Python + residential proxies)
  Maps  |  Job boards  |  Funding feeds
        |  POST clean JSON batches
        v
[ n8n Webhook: /ingest ] --> INSERT raw rows --> ( PostgreSQL: leads / companies )
                                                          |
        DB trigger / polling cron picks up status=DISCOVERED
        v
[ n8n WF-2  Enrichment Waterfall ]  Apollo -> Hunter -> BuiltWith -> PageSpeed -> Verify
        v   (status ENRICHED)
[ n8n WF-3  AI Qualify + Translate ] Basic LLM Chain -> route Segment A/B/C -> localize
        v   (status QUALIFIED + pitch ready)
[ n8n WF-4  Outreach Loader ]  push to Instantly/Smartlead  +  Cal.com link
        v
```

```
[ Reply / booking webhooks ] --> update lead status (REPLIED / BOOKED / WON / LOST)

Cross-cutting: every WF has an error branch -> Slack alert -> lead.status=ERROR (resumable)
```

ARCHITECT'S NOTE

Why this beats the 'single execution' monolith.

Each workflow is independently restartable and idempotent. If BuiltWith changes its schema, WF-2 fails on one node, alerts Slack, and parks that lead in ERROR - the sender, the calendar, and discovery keep running. The database state machine, not a call stack, holds the pipeline together.

2.2 Lead State Machine

Every lead is a row that walks a strict, auditable status enum. Illegal transitions are rejected at the database layer (trigger/check), so a bug can never, for example, queue an unqualified or suppressed lead for outreach.

TEXT · Lead status enum & legal transitions

```
DISCOVERED        -- raw row landed from a scraper, minimal fields
  -> ENRICHING    -- WF-2 claimed it, calling enrichment APIs
  -> ENRICHED     -- firmographics + decision-maker + tech stack + scores attached
  -> QUALIFYING   -- WF-3 LLM chain evaluating ICP fit
  -> QUALIFIED    -- fit; segment assigned (A/B/C); proceed
     | -> DISQUALIFIED   -- not a fit; archived with reason (terminal-ish)
  -> TRANSLATING  -- non-English region; localizing the pitch
  -> QUEUED_FOR_OUTREACH  -- verified + pitch ready; handed to sender
  -> CONTACTED    -- pushed into Instantly/Smartlead campaign, first send done
  -> REPLIED      -- prospect responded (positive/neutral/negative sub-state)
     | -> BOOKED -> WON | LOST     -- meeting + deal outcomes (feed back to qualifier)
Side states: SUPPRESSED (opt-out/bounce/GDPR), ERROR (node failure, resumable),
             COOLDOWN (contacted recently; locked N days)
```

2.3 Production PostgreSQL DDL Schema

Executable DDL. Normalized around companies (one per business) and leads (one per contactable person), with JSONB escape hatches for messy enrichment payloads and first-class fields for the intent signals the plan named: tech stack, Lighthouse mobile score, active job-post intent strings, and localized message text.

SQL · 01 - extensions & enumerated types

```
-- ============================================================
-- Extensions & enums
-- ============================================================
CREATE EXTENSION IF NOT EXISTS pgcrypto;     -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS citext;       -- case-insensitive email

CREATE TYPE lead_status AS ENUM (
  'DISCOVERED','ENRICHING','ENRICHED','QUALIFYING','QUALIFIED',
  'DISQUALIFIED','TRANSLATING','QUEUED_FOR_OUTREACH','CONTACTED',
  'REPLIED','BOOKED','WON','LOST','SUPPRESSED','ERROR','COOLDOWN'
);

CREATE TYPE icp_segment AS ENUM ('A_LEGACY_BRICK','B_FUNDED_STARTUP','C_LOWTECH_ECOM');
CREATE TYPE consent_basis AS ENUM ('LEGITIMATE_INTEREST','CONSENT','NONE');
```

```
CREATE TYPE region_code  AS ENUM ('US','EU','UK','GCC','CN','AU','OTHER');
```

SQL · 02 - companies

```
-- ============================================================
-- companies : one row per discovered business
-- ============================================================
CREATE TABLE companies (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  legal_name        TEXT        NOT NULL,
  domain            CITEXT      UNIQUE,            -- dedup key
  website_url       TEXT,
  phone             TEXT,
  region            region_code NOT NULL DEFAULT 'OTHER',
  country           TEXT,
  city              TEXT,
  niche             TEXT,                          -- e.g. 'logistics'
  employee_count    INT,                           -- enriched, for A/B gating
  gmaps_rating      NUMERIC(2,1),
  gmaps_reviews     INT,
  -- intent / qualification signals ---------------------------------
  tech_stack        TEXT[]      DEFAULT '{}',       -- ['React','Shopify',...]
  lighthouse_mobile INT,                            -- 0-100; <50 = Segment C
  lighthouse_lcp_ms INT,
  active_job_posts  JSONB       DEFAULT '[]',       -- [{title,url,seen_at,source}]
  intent_strings    TEXT[]      DEFAULT '{}',       -- ['hiring:Senior React','funding:seed']
  funding_stage     TEXT,
  funding_last_at   DATE,
  -- discovery bookkeeping ------------------------------------------
  source            TEXT        NOT NULL,           -- 'gmaps'|'jobboard'|'crunchbase'
  discovery_cell    TEXT,                           -- 'GCC|Riyadh|logistics'
  raw_payload       JSONB,                          -- untouched scraper output
  first_seen_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_verified_at  TIMESTAMPTZ,
  cooldown_until    TIMESTAMPTZ,                    -- per-company contact lock
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_companies_region_niche ON companies (region, niche);
CREATE INDEX idx_companies_techstack    ON companies USING GIN (tech_stack);
CREATE INDEX idx_companies_jobs         ON companies USING GIN (active_job_posts);
```

SQL · 03 - leads (the state-machine table)

```
-- ============================================================
-- leads : one row per contactable person at a company
-- ============================================================
CREATE TABLE leads (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id        UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  full_name         TEXT,
  title             TEXT,                           -- 'Founder','CTO'
  email             CITEXT,
  email_status      TEXT,                           -- 'valid'|'risky'|'invalid'|'catch_all'
  email_verified_at TIMESTAMPTZ,
  linkedin_url      TEXT,
  -- state machine --------------------------------------------------
  status            lead_status NOT NULL DEFAULT 'DISCOVERED',
  segment           icp_segment,
  qualify_reason    TEXT,                           -- LLM 1-sentence rationale
```

```
  qualify_score      NUMERIC(4,2),
  disqualify_reason  TEXT,
  -- localized messaging ----------------------------------------
  pitch_lang         TEXT        DEFAULT 'en',       -- 'en'|'ar'|'zh'
  pitch_subject      TEXT,
  pitch_body         TEXT,                           -- final, localized outreach copy
  pitch_localized    BOOLEAN     DEFAULT FALSE,
  -- compliance -------------------------------------------------
  consent            consent_basis NOT NULL DEFAULT 'NONE',
  lia_reference      TEXT,                           -- legitimate-interest assessment id
  suppressed_at      TIMESTAMPTZ,
  suppression_reason TEXT,                           -- 'optout'|'bounce'|'gdpr'|'complaint'
  erasure_requested_at TIMESTAMPTZ,
  -- outreach linkage -------------------------------------------
  outreach_provider  TEXT,                           -- 'instantly'|'smartlead'
  campaign_id        TEXT,
  sending_domain     TEXT,                           -- burner domain used
  last_contacted_at  TIMESTAMPTZ,
  replied_at         TIMESTAMPTZ,
  reply_sentiment    TEXT,                           -- 'positive'|'neutral'|'negative'
  booked_at          TIMESTAMPTZ,
  outcome            TEXT,                           -- 'won'|'lost'|null (qualifier feedback)
  error_detail       TEXT,
  enrichment         JSONB       DEFAULT '{}',       -- raw API payloads
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX idx_leads_email_live ON leads (email)
  WHERE email IS NOT NULL AND suppressed_at IS NULL;   -- no dup live emails
CREATE INDEX idx_leads_status  ON leads (status);
CREATE INDEX idx_leads_segment ON leads (segment);
```

SQL · 04 - suppression, depletion & audit

```
-- ============================================================
-- suppression_list : irreversible global block (GDPR + CAN-SPAM)
-- ============================================================
CREATE TABLE suppression_list (
  email_hash   TEXT PRIMARY KEY,        -- sha256(lower(email)) - store hash, not PII
  domain       CITEXT,
  reason       TEXT NOT NULL,           -- 'optout'|'bounce'|'complaint'|'gdpr'|'trap'
  added_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- discovery_cells : depletion accounting for market-fatigue control
CREATE TABLE discovery_cells (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  region         region_code NOT NULL,
  city           TEXT NOT NULL,
  niche          TEXT NOT NULL,
  total_seen     INT DEFAULT 0,
  new_last_run   INT DEFAULT 0,
  contacted      INT DEFAULT 0,
  saturation     NUMERIC(5,4) DEFAULT 0,   -- new_last_run / NULLIF(total_seen,0)
  depleted       BOOLEAN DEFAULT FALSE,
  last_run_at    TIMESTAMPTZ,
  cooldown_until TIMESTAMPTZ,
  UNIQUE (region, city, niche)
);

-- events : append-only audit of every status transition (funnel math)
```

```
CREATE TABLE lead_events (
  id         BIGSERIAL PRIMARY KEY,
  lead_id    UUID REFERENCES leads(id) ON DELETE CASCADE,
  from_status lead_status,
  to_status   lead_status NOT NULL,
  workflow    TEXT,                       -- 'WF-2','WF-3','WF-4'
  detail      JSONB,
  at          TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_events_lead ON lead_events (lead_id, at);
```

SQL · 05 - DB-layer compliance guardrail

```
-- ============================================================
-- Guardrails: enforce hygiene at the database layer
-- ============================================================
-- 1) Block queueing a suppressed or unverified lead for outreach
CREATE OR REPLACE FUNCTION enforce_outreach_gate() RETURNS trigger AS $$
BEGIN
  IF NEW.status = 'QUEUED_FOR_OUTREACH' THEN
    IF NEW.suppressed_at IS NOT NULL THEN
      RAISE EXCEPTION 'Lead % is suppressed; cannot queue', NEW.id;
    END IF;
    IF NEW.email_status IS DISTINCT FROM 'valid' THEN
      RAISE EXCEPTION 'Lead % email not verified valid', NEW.id;
    END IF;
    IF EXISTS (SELECT 1 FROM suppression_list
               WHERE email_hash = encode(digest(lower(NEW.email::text),'sha256'),'hex')) THEN
      RAISE EXCEPTION 'Lead % on global suppression list', NEW.id;
    END IF;
  END IF;
  NEW.updated_at := now();
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_outreach_gate BEFORE INSERT OR UPDATE ON leads
  FOR EACH ROW EXECUTE FUNCTION enforce_outreach_gate();
```

SECTION 03

High-Risk Vulnerabilities & Anti-Ban Countermeasures

Two structural risks can sink this project: getting your platform accounts permanently banned, and getting your sending domains globally blocklisted. This section tears apart the riskiest parts of the plan and replaces them with compliant, high-velocity equivalents.

3.1 Upwork / Fiverr / LinkedIn Automation: Tear-Down

The plan's idea of bots that log into Upwork/Fiverr/LinkedIn accounts and send automated messages is the single highest-risk component and must be removed from scope. The risk/reward is catastrophic:

- **These platforms fingerprint the browser, not just the IP.** Canvas, WebGL, font, and behavioral fingerprints defeat naive proxy rotation. Selenium/Puppeteer automation signatures and form-submission cadence are detected fast.
- **The asset you burn is irreplaceable.** An Upwork agency account carries your job history, reviews, and Rising-Talent/Top-Rated status. A ban erases years of reputation - you cannot just spin up granjurtech.com #2.
- **Detection is often silent.** Upwork commonly shadow-throttles (your proposals stop being seen) before an outright ban, so you can burn the account without even knowing.
- **The ToS violation is unambiguous.** Automated messaging/scraping is explicitly prohibited; there is no compliant way to do it at volume.

RISK

Verdict: delete automated platform messaging entirely. Keep only read-only intent discovery.

Reading public job posts to detect intent is defensible and low-footprint. Acting on the platform (messaging, connecting, applying) as a bot is not. Separate the two permanently: bots discover, humans act.

The replacement: a Human-in-the-Loop (HITL) Slack/Discord queue

Convert the risky "auto-apply / auto-message" concept into a high-velocity human workflow. The bot does the tedious 95% (find the post, parse it, score fit, draft the message, attach the apply link); a human does the compliant 5% (one click to send from a real logged-in session). You keep the speed and lose the ban risk.

JSON · HITL Slack Block Kit payload + callbacks

```
// n8n WF: intent-to-Slack (Human-in-the-Loop)
// Trigger: new company.active_job_posts row matching a target keyword
{
  "channel": "#upwork-hot-leads",
  "blocks": [
    { "type": "header", "text": "New React job - strong fit (score 0.86)" },
    { "type": "section", "fields": [
        "*Company:* Nimbus Logistics",  "*Region:* GCC / Riyadh",
        "*Signal:* hiring Senior React Dev", "*Stack:* React, Node, AWS",
```

```
          "*Employees:* 60 (Segment B)" ] },
      { "type": "section", "text": "*Suggested DM (review before sending):*\n       Hi - saw your Senior React role.
      { "type": "actions", "elements": [
          { "type":"button", "text":"Open job post", "url":"https://...", "style":"primary" },
          { "type":"button", "text":"Copy draft DM", "action_id":"copy_draft" },
          { "type":"button", "text":"Mark handled", "action_id":"mark_handled" },
          { "type":"button", "text":"Not a fit", "action_id":"dismiss" } ] }
    ]
}
// 'Mark handled' -> Slack interactivity webhook -> n8n -> leads.status='CONTACTED'
// 'Not a fit'    -> leads.status='DISQUALIFIED', feeds qualifier training set
```

FIX

Channel doctrine: bots automate Email + LinkedIn-soft-touch; humans handle Upwork/Fiverr.

Email (off-platform, your infrastructure) is your only fully-automated outbound channel. LinkedIn connection requests stay manual or via a vetted compliant tool at human-like volume. Upwork/Fiverr are 100% HITL via the Slack queue. This preserves every account while keeping operator throughput high.

3.2 Scraper Footprint Optimization

Rule: minimize the custom, ban-prone surface. Only true discovery scraping needs proxies and anti-fingerprinting. Everything that can be a clean outbound API call should be one - run inside n8n, observable and rate-limited.

Component	Build mode	Network posture	Why
Script 1 - Google Maps discovery	Custom headless (proxied)	Rotating residential proxies + jitter	Maps blocks datacenter IPs & fast queries
Script 1 - Job board discovery	Custom headless / RSS (proxied)	Residential proxies; prefer RSS/JSON endpoints	HTML scraping is brittle & rate-limited
Decision-maker emails	Apollo / Hunter API (n8n)	Plain outbound HTTPS, API key	Vendor already holds verified records
Firmographics / employee count	Apollo / Clay API (n8n)	Plain outbound HTTPS	No need to scrape LinkedIn yourself
Tech stack	BuiltWith / Wappalyzer API (n8n)	Plain outbound HTTPS	Licensed dataset, zero ban risk
Mobile performance	Google PageSpeed API (n8n)	Plain outbound HTTPS	Official Google API; just needs a key
Email verification	ZeroBounce / NeverBounce API	Plain outbound HTTPS	Trap/catch-all detection as a service
LinkedIn data (if ever needed)	Vetted provider API (PhantomBuster/Proxycurl)	Via vendor, never your own bot	Offload the fingerprinting risk entirely

Hardening the only custom surface (Script 1)

- **Residential, rotating, geo-matched proxies.** Match proxy geo to the target region (GCC traffic from GCC exit nodes). Datacenter IPs are flagged instantly on Maps.
- **Human-like pacing, not 24/7 hammering.** Randomized delays (multi-second), batch by city x niche block, hard daily caps per proxy pool. The source plan's own correction - run Maps in batches, not

continuously - is right; enforce it in the scheduler.

- **Anti-fingerprint hygiene.** Real headless-evasion (stealth plugin), rotated user-agents consistent with the IP, realistic viewport/headers, no two requests with an identical fingerprint.
- **Fail closed and quietly.** On CAPTCHA or block detection, back off exponentially and alert Slack - never retry-storm, which is what converts a soft rate-limit into a hard ban.
- **Keep payloads thin.** Script 1 extracts only identity + a few raw fields, then POSTs JSON to n8n. All heavy enrichment happens later via APIs - so a Script-1 failure costs you almost nothing and never blocks the rest of the pipeline.

ARCHITECT'S NOTE

Net effect on footprint.

Of ~8 data-gathering concerns in the plan, exactly one or two (Maps + job discovery) remain custom and proxied. The other six become boring, restartable API nodes. That is the whole game: a tiny attack surface you can harden completely, behind a large orchestration layer you can observe completely.

SECTION 04

Step-by-Step n8n Workflow & Script Breakdown

Concrete engineering parameters, triggers, webhook schemas, and edge cases for the four modules. Each phase is an independently restartable n8n workflow reading and writing the Section 2 state machine.

Phase 1 - Custom Discovery Engine (Script 1)

Responsibility & operating mode

Lightweight headless scrapers (Node/Python) behind residential proxies. They do one job: find business entities, package them as clean JSON, POST to an n8n webhook, and move on. No enrichment, no scoring, no sending. Scheduled in batches by city x niche cell - never an unthrottled 24/7 loop.

- **Trigger:** External cron on the scraper host (e.g. every 2-4h per region), reading the next non-depleted cell from discovery_cells ordered by oldest last_run_at.
- **Batch size:** ~40-80 entities per cell per run; hard daily cap per proxy pool; randomized inter-request delay.
- **Output:** One HTTP POST of a JSON batch to the n8n ingestion webhook, idempotency key per entity (domain or maps_place_id) so re-runs never duplicate.

JSON · Phase-1 ingestion webhook schema

```
POST https://n8n.granjur.internal/webhook/ingest
Headers: { "X-Api-Key": "<scraper-shared-secret>", "Content-Type":"application/json" }
{
  "source": "gmaps",
  "discovery_cell": "GCC|Riyadh|logistics",
  "scraped_at": "2026-06-24T09:12:00Z",
  "batch": [
    {
      "dedup_key": "place_ChIJ_xxx",          // maps place id or domain
      "legal_name": "Nimbus Logistics Co.",
      "domain": "nimbuslogistics.sa",
      "website_url": "https://nimbuslogistics.sa",
      "phone": "+966-11-xxx",
      "region": "GCC", "country":"SA", "city":"Riyadh", "niche":"logistics",
      "gmaps_rating": 4.6, "gmaps_reviews": 213,
      "raw_payload": { /* untouched scraper object */ }
    }
  ]
}
```

n8n ingestion workflow & edge cases

- **Validate + auth:** Reject batches missing X-Api-Key or with malformed entities (return 207 with per-row errors so the scraper can log them).
- **Upsert, don't insert:** ON CONFLICT (domain) / (dedup_key) DO UPDATE last-seen fields; increment discovery_cells.total_seen / new_last_run for saturation tracking.
- **New rows land as status='DISCOVERED'** and emit a lead_events row; this is the only signal WF-2 needs.

- **Edge - depletion:** After upsert, recompute saturation = new_last_run/total_seen; if < 0.05 for two runs, set depleted=TRUE and set cooldown_until so the scheduler rotates to an adjacent cell (Section 1.4).
- **Edge - cooldown collision:** If an incoming company.cooldown_until is in the future, update firmographics but do NOT create a new outreach lead.

Phase 2 - n8n Data Enrichment Waterfall (Script 2)

Responsibility

Claim DISCOVERED rows and walk them through a fault-tolerant chain of outbound API calls, attaching decision-makers, firmographics, tech stack, mobile score, and email verification. The defining requirement: one slow or failing API must never halt the pipeline.

- **Trigger:** Schedule/poll every few minutes: SELECT ... WHERE status='DISCOVERED' LIMIT 25 FOR UPDATE SKIP LOCKED (queue-mode workers, no double-processing).
- **Claim:** Set status='ENRICHING' immediately so a crash leaves a recoverable state, not a lost lead.

Waterfall order & robust fallback paths

1. Apollo.io (primary) -> decision-maker name, title, email, employee_count. On 429/timeout, do NOT fail the lead: catch in an n8n Error branch, mark this node 'skipped', continue.
2. Hunter.io (fallback) -> only invoked if Apollo returned no email; domain-search for a pattern-matched address.
3. BuiltWith / Wappalyzer -> tech_stack[]. Independent of the email path; failure just leaves tech_stack empty (qualifier handles nulls).
4. Google PageSpeed -> lighthouse_mobile + LCP. Cache by domain for 30 days to spare quota.
5. Email verification (ZeroBounce/NeverBounce) -> set email_status. 'invalid' -> never send; 'catch_all/risky' -> flag, route to manual or skip.

FIX

Fault-tolerance pattern: per-node try/catch + a partial-success rule, not all-or-nothing.
Each API node has its own Error output wired to a 'record failure' set-node that writes to leads.enrichment.errors[] and continues. A lead reaches ENRICHED if it has at least a verified email OR a strong intent signal; otherwise it parks in ERROR for a retry job - it never blocks siblings. Use n8n retryOnFail (3x, exponential backoff) on transient 5xx/429, and a circuit-breaker: if an API errors >X% in 10 min, pause that node and Slack-alert.

SQL · Phase-2 atomic claim (no double-processing)

```sql
-- Worker claim query (queue-mode safe)
UPDATE leads SET status='ENRICHING', updated_at=now()
WHERE id IN (
  SELECT id FROM leads
  WHERE status='DISCOVERED'
  ORDER BY created_at
  FOR UPDATE SKIP LOCKED
  LIMIT 25
)
RETURNING id, company_id;
```

Phase 3 - n8n Advanced AI Qualification & Translation (Script 3)

Responsibility

Take ENRICHED leads, decide fit, assign one of three segments, generate a trigger-specific pitch, and localize it where required - all without producing spam-shaped text. Uses n8n's Basic LLM Chain / AI Agent nodes with a structured-output (JSON) contract so routing is deterministic.

Qualification chain

- **Input context:** company description, employee_count, tech_stack, active_job_posts, lighthouse_mobile, region - assembled into a compact prompt.
- **Structured output:** force JSON {qualified: bool, segment: 'A'|'B'|'C', score: 0-1, reason: string, trigger: string}. n8n's IF/Switch routes on segment; reason+score are stored for the feedback loop (Section 1.1, Tip 4).
- **Guardrails:** hard pre-filters in SQL before the LLM (employee_count gating: <15 or >150 for Segment A auto-disqualifies) to save tokens and stop the model overriding firm business rules.

Segment	Who	Primary trigger -> pitch angle
A - Legacy Brick & Mortar	Mid-size local, 20-150 staff	High rating + weak web/booking -> digital transformation
C - Low-Tech E-commerce	D2C on basic Shopify/Woo	Lighthouse mobile < 50 -> quantified revenue-loss fix

TEXT · Phase-3 qualifier prompt contract

```
SYSTEM (qualifier):
You are Granjur's lead-qualification analyst. Granjur sells custom dev, MVPs, and
staff augmentation. Given the company profile, decide fit and segment. Output ONLY
valid JSON. Disqualify single-person firms and >150-employee firms with internal IT.

USER (interpolated by n8n):
{ name, country, employee_count, tech_stack, active_job_posts, lighthouse_mobile, desc }

EXPECTED OUTPUT:
{ "qualified": true, "segment": "B", "score": 0.86,
  "trigger": "hiring Senior React Developer",
  "reason": "Series-A, 60 staff, actively hiring React - prime staff-aug fit." }
```

Translation / localization layer (geo-routed, non-spammy)

- **Branch on region:** GCC -> Gulf-corporate Arabic; CN -> Simplified Chinese; else keep English. Only QUALIFIED leads are translated, conserving tokens.
- **Translate intent, not strings:** Prompt the LLM to rewrite the pitch as a native business writer would - localized idiom, honorifics, RTL-aware - NOT a literal Google Translate pass (which reads as spam and tanks deliverability).
- **Persist before send:** Write pitch_subject/pitch_body/pitch_lang and set pitch_localized=TRUE on the lead so Phase 4 sends pristine, pre-localized copy with zero runtime translation lag.
- **Anti-spam shaping:** Enforce no spam-trigger tokens, no ALL-CAPS, balanced text/link ratio, a single soft CTA, and a personalized first line referencing the trigger.

WATCH-OUT

RTL & rendering caveat for the build, not just the copy.

Arabic must be stored as proper Unicode and sent through a mail platform that renders RTL correctly; verify in Instantly/Smartlead preview before campaign launch. Keep an English fallback subject for clients whose clients read English in business contexts (common in the GCC).

Phase 4 - Outreach Engine & Calendar Integration (Script 4)

Responsibility

Do NOT hand-roll timezone or cadence logic. n8n is purely a data loader: it pushes verified, localized, QUALIFIED leads into Instantly.ai or Smartlead.io and lets the platform own warmup, timezone matching, follow-ups, and A/B testing. Calendar booking is delegated to Cal.com/Calendly so overseas slots map to PKT working hours.

- **Trigger:** status='QUALIFIED' AND email_status='valid' AND pitch_body present AND not suppressed. The DB trigger from Section 2.3 is the final safety net.
- **Routing:** Map segment + region -> a specific campaign id (and a specific burner sending domain) so reporting and warmup stay isolated per segment.
- **Idempotency:** Store campaign_id + provider id on the lead; never re-push the same lead; set status='QUEUED_FOR_OUTREACH' then 'CONTACTED' on provider ack.

JSON · Phase-4 outreach loader payload

```
POST https://api.instantly.ai/api/v2/leads     (or Smartlead /leads)
{
  "campaign_id": "seg-B-gcc-2026",
  "email": "founder@nimbuslogistics.sa",
  "first_name": "Khalid",
  "company_name": "Nimbus Logistics",
  "sending_account": "outreach@getgranjur.com",   // burner, never granjur.com
  "personalization": {
     "subject": "<localized pitch_subject>",
     "body": "<localized pitch_body>",
     "trigger": "hiring Senior React Developer",
     "booking_link": "https://cal.com/granjur/intro?tz=auto"
  },
  "custom_vars": { "lead_id": "<uuid>", "segment": "B", "region": "GCC" }
}
// Provider handles: warmup, timezone send-window, follow-up steps, A/B variants.
```

Calendar integration - PKT-aware without custom timezone code

- **Configure availability once, in PKT.** In Cal.com/Calendly set Granjur availability to real PKT working hours (e.g. 14:00-22:00 PKT). The tool translates and shows only overlapping slots to the prospect in THEIR timezone.
- **This kills the 'dead zone' bug.** A New York prospect can no longer book an 11:00 EST slot that lands at 20:00-21:00 PKT unless that window is inside your configured PKT availability - the API enforces the overlap for you.
- **Close the loop:** Cal.com booking webhook -> n8n -> leads.status='BOOKED', booked_at set; a no-show/again later flips to LOST. WON/LOST outcomes feed the qualifier retraining set.

FIX

Follow-up & reply handling stays on the platform, mirrored to the DB.

Let Instantly/Smartlead run the 4-day-then-LinkedIn-soft-touch waterfall from the source plan. Mirror reply + bounce + unsubscribe webhooks back into n8n: positive -> REPLIED(positive) + Slack ping a human; bounce -> SUPPRESSED + suppression_list insert; unsubscribe -> SUPPRESSED(gdpr/optout). The database, not the email tool, remains the system of record.

Implementation Sequencing (Recommended Order)

1. Stand up Postgres + the Section 2 schema and the suppression/guardrail triggers FIRST - compliance and state model before any sending.
2. Build Phase 1 discovery + the n8n ingestion webhook; prove clean, deduped data lands as DISCOVERED with depletion accounting working.
3. Build Phase 2 enrichment waterfall with full fault-tolerance and email verification; do not send anything yet.
4. Build Phase 3 qualifier + localization with structured output and SQL pre-filters; hand-review the first few hundred decisions to calibrate the prompt.
5. Warm 3-5 burner domains/mailboxes for 2-3 weeks in parallel with steps 1-4 (warmup is a lead-time item, start it day one).
6. Wire Phase 4 to Instantly/Smartlead + Cal.com last; launch one segment, one region, low volume; watch bounce/complaint rates before scaling.
7. Turn on the won/lost feedback loop and the weekly funnel-math rollup; only then scale volume and add regions.

ARCHITECT'S NOTE

The one-sentence thesis.

Keep the custom, ban-prone surface microscopic (discovery only), make the database the compliant source of truth for a strict lead state machine, let n8n orchestrate fault-tolerant API calls for everything else, and let specialist platforms own the regulated last mile of sending and scheduling. That is how this pipeline compounds instead of burning your domains and your market.